# How a Python Function Really Works

Week 01 - Logic | Class 04 - Functions | Companion note for EX10

Mental model

Core idea: defining a function creates a reusable tool; calling it executes that tool with concrete data.

## 1. What happens when Python reaches `def`

When Python executes a `def` statement, it creates a function object and binds that object to the function name. The body is not executed yet.

```
def calculate_rocket_delta_v(isp, m0, mf, g0=9.80665):
    ...
```

After this definition, Python knows that a function named calculate_rocket_delta_v exists. Delta-v is calculated only when the function is called.

## 2. Parameters and arguments

Parameters are the names in the definition. Arguments are the values or objects supplied in a call.

| Definition | Call | What happens |
|---|---|---|
| isp | 250 | isp receives 250 |
| m0 | 500000 | m0 receives 500000 |
| mf | 250000 | mf receives 250000 |
| g0=9.80665 | (omitted) | default value is used |

## 3. Your observation about names: positional vs keyword

The key nuance is: positional arguments in Python do NOT need to have the same variable names as the parameters. Only their order matters.

```
engine_isp = 250
wet_mass = 500000
dry_mass = 250000

dv = calculate_rocket_delta_v(engine_isp, wet_mass, dry_mass)
```

This works even though the outside variables are named differently. Python binds by position: first to `isp`, second to `m0`, third to `mf`.

With keyword arguments, however, the name on the left must match the parameter name exactly:

```
dv = calculate_rocket_delta_v(
    isp=engine_isp,
    m0=wet_mass,
    mf=dry_mass
)
```

Practical rule

In a keyword call, isp=, m0= and mf= are part of the function interface. On the right side of =, you can use any variable or expression.

# 4. What about C/C++?

Your memory is correct. Normal C and C++ function calls are positional: caller variable names are irrelevant. Arguments are matched by order and type. Parameter names are not part of the function signature.

```
// C++
double rocketDv(double isp, double m0, double mf);

double engineIsp = 250;
double wetMass = 500000;
double dryMass = 250000;

double dv = rocketDv(engineIsp, wetMass, dryMass);
```

Python supports this positional style too, but adds a useful extra feature: keyword arguments.

# 5. What exactly happens during a function call

| | |
|---|---|
| 1 | Python evaluates the argument expressions. |
| 2 | It binds them to parameters by position, keyword, or defaults. |
| 3 | It creates a new local execution context for that call (a frame). |
| 4 | It executes the function body from top to bottom. |
| 5 | If return is reached, the function ends immediately. |
| 6 | The returned object goes back to the exact call site. |
| 7 | The main program can store that object in a variable. |

```
delta_v = calculate_rocket_delta_v(250, 500000, 250000)
```

You can mentally read it as:

**delta_v <- object returned by calculate_rocket_delta_v(... )**

# 6. Scope: parameters and local variables

Inside the function, `isp`, `m0`, `mf`, `mass_ratio` and `delta_v` are local to that call. A new call creates a new set of local values.

> Reuse
> That is why the same function can be reused hundreds of times: every call has its own local state.

# 7. return, return None and immediate exit

```
if mf <= 0:
    return None

mass_ratio = m0 / mf
```

If `mf <= 0`, Python executes `return None` and exits immediately. Later lines in that call are skipped. The caller receives None and decides what to do.

# 8. Default parameters

```
def calculate_rocket_delta_v(isp, m0, mf, g0=9.80665):
```

The first three parameters are required. `g0` is optional because it has a default. If omitted, 9.80665 is used; if supplied, the provided argument replaces the default.

# 9. Useful call rules

| Rule | Meaning |
|---|---|
| Positional | Bound by order. External variable names do not matter. |
| Keyword | The name on the left of = must match the parameter. |
| Default | Used only when that parameter receives no other argument. |
| Mixing | Positional arguments must come before keyword arguments. |
| Duplicate | You cannot provide the same parameter twice. |
| Case | Python distinguishes isp from Isp: they are different names. |

# 10. Review of your EX10

Your exercise correctly demonstrates positional calls, keyword arguments, a default for g0, early validation with return None, and function reuse.

> Model requirement
>
> Model decision: your current function rejects `m0 == mf` because it uses `m0 <= mf`. If you want the no-propellant-consumption case to be mathematically valid, change it to `m0 < mf`; then `m0 == mf` naturally returns delta-v = 0. If you prefer to define it as an invalid burn, keep the current code and document that requirement.

For scalar values, `math.log` is enough and avoids a NumPy dependency. `np.log` is not wrong; it becomes especially useful once we work with arrays.

# 11. Learning Tracker

| Concept | What to remember |
|---|---|
| Function definition | def creates a function; it does not execute the body. |
| Parameters | Internal names that receive values. |
| Arguments | Values supplied at call time. |
| Positional binding | Binding by order. |
| Keyword binding | Binding by name. |
| Default parameter | Fallback value when the argument is omitted. |
| Local scope | Each call creates its own local variables. |
| return | Ends the function and returns an object. |
| return None | Early exit without a valid numeric result. |
| Reuse | The same function can be called for many cases. |

Control question: if you see f(a=external_value), which name must match the function parameter?
Answer: a. The name external_value can be anything.